Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 10
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest



Cuprins

- Arhitecturi MIMD – Caracteristici
- Sisteme Multiprocesor / Multicalculator
- Sisteme cu Memorie Partajata
- Sisteme cu Memorie Distribuita
- Sisteme Strans / Slab Cuplate
- Accesul la Resursele Locale – Switch-ul S
- Structura si Functiile K_{MAP}
- Arhitectura Procesorului de Mapare P_{MAP}
- Comunicarea in Cadrul unui Cluster
- Comunicarea Intercluster



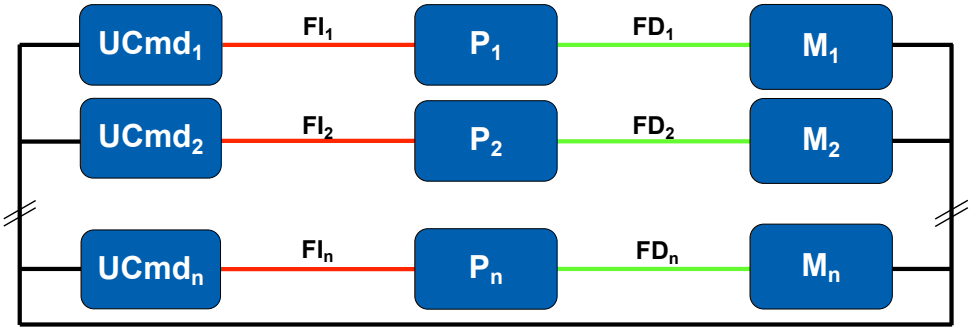
MiMD

Arhitecturi MIMD

MIMD

- MIMD pot comunica: (P-P sau P-M)
- Toate Procesoarele participa la acelasi program
- Mult mail des folosite ca SIMD
- Mod programare:
  - Shared memory (strans cuplate) – Memorie Partajata (e.g. OpenMP)
  - Distributed memory (slab cuplate) – Transfer de Mesaje (e.g. MPI)

MIMD = n FI & n FD



Arhitecturi MIMD – Caracteristici

- Structura & arhitectura procesoarelor e eterogena
- Procesoarele pot functiona autonom ca un sistem universal
  - Pot prelua task-urile altor procesoare din sistem
  - Astfel se previne degradarea performantelor sistemului
- Organizarea memoriei
  - Fiecare procesor ar trebui sa aiba acces la toata memoria sistemului, dar…
    - Capacitatea de adresare a fiecarui procesor e mai mica ca dimensiunea memoriei → mecanisme speciale
  - Trebuie asigurata protectia memoriei pentru asigurarea integritatii datelor din sistem



Arhitecturi MIMD – Caracteristici

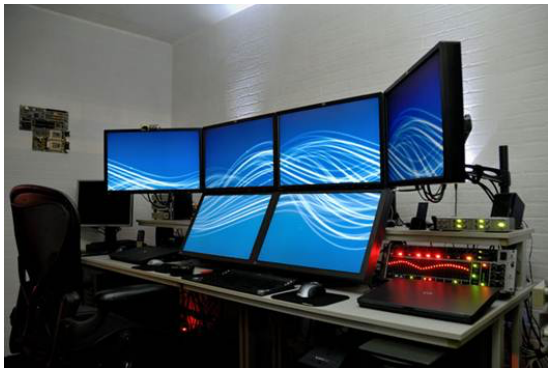
- Modularitatea presupune
  - Expandabilitate
    - Numarul de procesoare poate fi modificat fara a fi necesara reconfigurarea sistemului
  - Reconfigurabilitate
    - Structurala
      - Procesul de refacere a starii sistemului la aparitia unor defecte → se realizeaza prin distribuirea structurala a functiilor
      - Permite preluarea sarcinilor unui procesor de catre alte procesoare din sistem
    - Arhitecturala: schimbarea caracteristicilor de baza
      - Dimensiunea registrilor de memorie
      - Lungimea cuvantului
      - Structura UAL



Arhitecturi MIMD – Probleme

- Decompozitia algoritmilor
  - Manual – la conceptia algoritmilor
  - Automat – prin limbaje de programare concurenta
- Planificarea pentru executie a task-urilor si maparea pe structura de procesoare utilizata
- Sincronizarea proceselor/task-urilor astfel incat sa nu se ajunga la blocarea sistemului
- Siguranta in functionare e asigurata de
  - Disponibilitate: capacitatea sistemului de a raspunde cererilor utilizatorului
  - Integritate: capacitatea sistemului de a mentine consistenta starii in conditiile aparitiilor unor defecte





System Multiprocesor / Multicalculator

Clasificarea Structurilor MIMD

- Din punct de vedere arhitectural, structurile MIMD se pot imparti in doua mari categorii:
  - Sisteme multiprocesor:
    - Sistem de calcul unitar unde o colectie de procesoare autonome functioneaza concurent si coopereaza la realizarea unei functii logice si de calcul de sub controlul unui “executiv” unic
  - Sisteme multicalculator:
    - Mai multe calculatoare autonome, interconectate, ce colaboreaza la realizarea unor task-uri din cadrul unei lucrari complexe



Sisteme Multiprocesor

- Au un singur spatiu de adresare - comunicare prin shared memory
- Se impart in sisteme:
  - Cu memorie distribuita / scalabilitate:
    - Gestiune dinamica a accesului la memoria distribuita
    - Gestiune statica a accesului la memoria distribuita
  - Cu memorie centrala / nu exista scalabilitate:
    - Comunicarea se face prin retele de comutatie (e.g. crossbar switch)
    - Conexiuni de tip magistrala (bus) multiplu



Sisteme Multicalculator

- Au spatii multiple de adresare - comunicare prin transfer de mesaje
- Se impart in sisteme:
  - Multicalculator distribuite:
    - Mesh – plasa de interconectare P-M, NEC…
    - Conexiuni Butterfly (Perfect Shuffle) – CM5
    - Hypercube – la sistemele NCUBE, BlueGene…
    - Retele LAN rapide: Myrinet, ATM, GigaEthernet, InfiniBand, MPLS …
  - Multicalculator centralizate

Clasificarea Structurilor MIMD

- Din punct de vedere al accesului la memorie, structurile MIMD sunt:
  - Sisteme cu memorie partajata - Shared Memory



  - Sisteme cu memorie distribuita - Distributed Memory





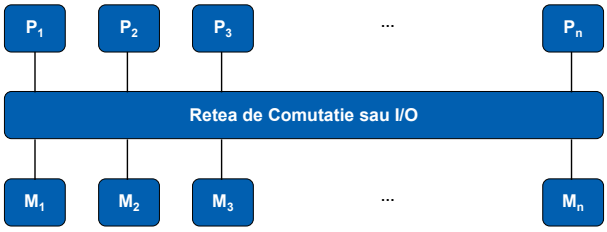
Sisteme cu Memorie Partajata

Sisteme cu Memorie Partajata

- Sistemele cu memorie partajata (Shared Memory) pot fi implementate in una din urmatoarele configuratii:
  - UMA = Uniform Memory Access
  - NUMA = Non-Uniform Memory Access
    - CC-NUMA = Cache-Coherent NUMA
  - COMA = Cache-Only Memory Architecture

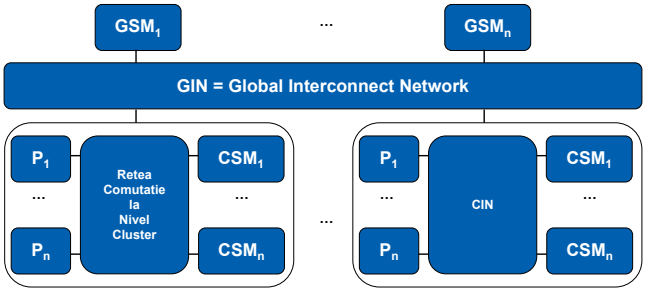
Structura UMA

- UMA = Uniform Memory Access
- Accesul este unic: orice procesor se poate conecta la un modul de memorie in mod uniform
- Reteaua de comunicatie (I/O) poate fi de orice tip



Structura NUMA

- CSM = Cluster Shared Memory
- CIN = Cluster Interconnect Network
- GSM = Global Shared Memory
- GIN = retea de comutare globala la care se conecteaza clustere formate din procesoare
- Accesul la memorii depinde de modulul unde trebuie efectuat accesul (CSM_i sau GSM_j)



Structura CC-NUMA

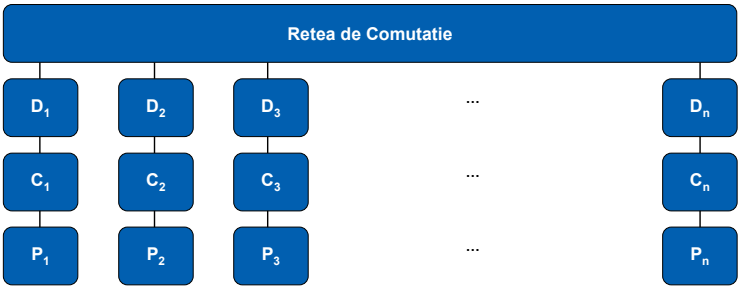
- CC-NUMA = Cache Coherent NUMA
- Coerenta accesului la memorie e mai bine stabilita ca la NUMA:
  - Silicon Graphics - Origin 2000:
    - 2 procesoare pe nod
    - Acces divizat la componentele nodului printr-un Bus intern
    - Hub de conectare asigura accesul partajat la Cache-ul nodului → coerenta cache-ului
  - Sequent NUMA-Q - structuri de noduri interconectate cu structura inelara:
    - Standardul IEEE – SCI = Scalable Coherent Interface – structura de comunicare "rapida" (1GB/s in the 00's)
  - Standford DASH (Directory Architecture for SHared memory) – 4 MIPS R3000 Procs





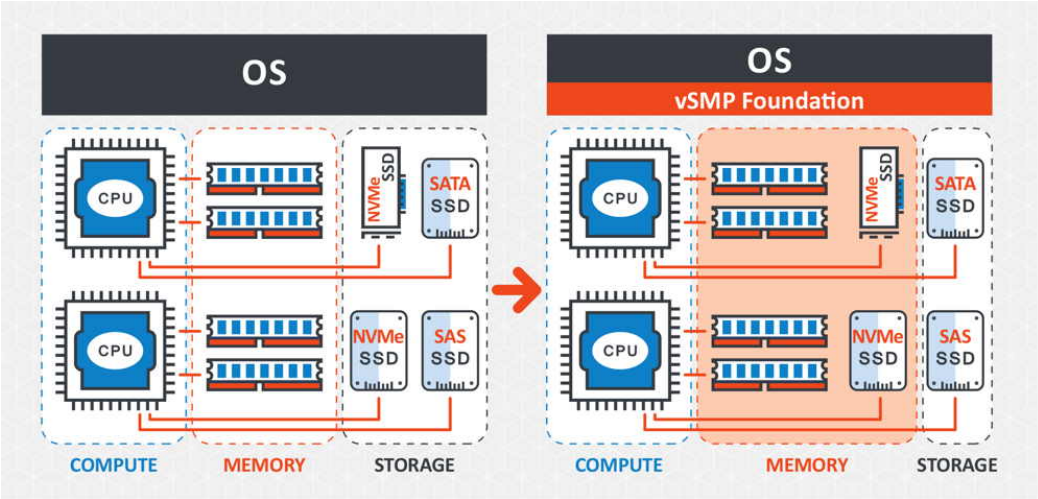
Structura COMA

- Procesoarele nu au acces direct
  - La propria zona de date
  - La cea a celorlate procesoare
- Procesoarele au acces direct doar la Cache! → Cache Only
  - D_i = Memoria de Date
  - C_i = Memoria Cache
  - P_i = Procesoare

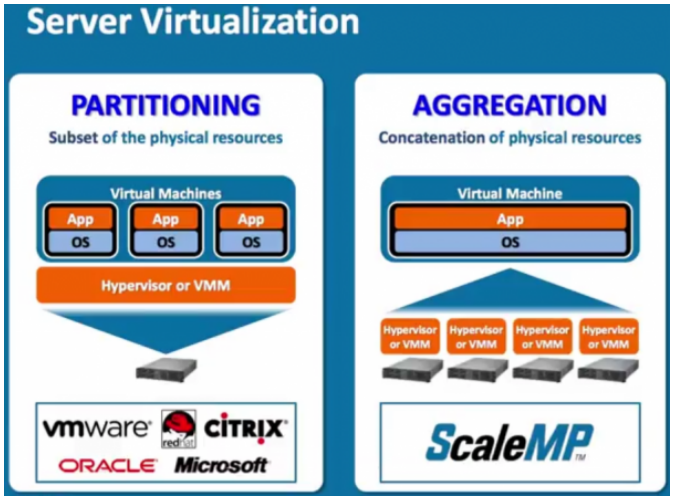


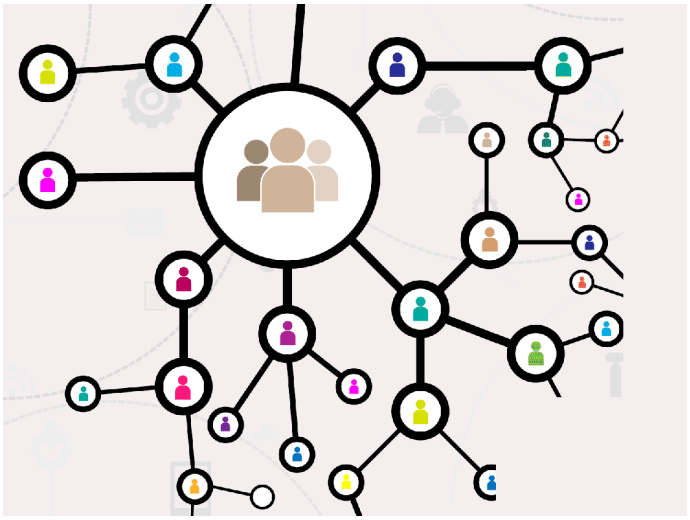


Exemplu de SW COMA



Server Virtualization

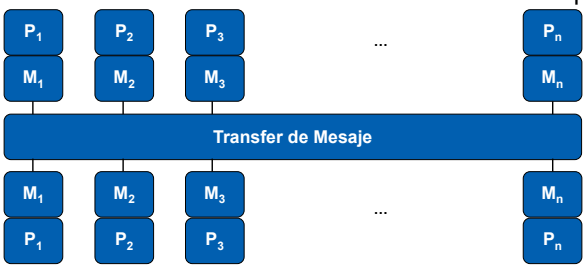




Sisteme cu Memorie Distribuita

Sisteme cu Memorie Distribuita

- Accesul se face prin transfer de mesaje
- Sistemele au la baza mai multe noduri legate printr-o retea de comutatie de mesaje (nu neaparat fizica)
- Conexiunea este statica, punct la punct intre noduri
- Memoria locala este adresabila direct doar de catre procesorul local



Exemplu @cs.pub.ro

- SUN Enterprise 10000 – Starfire (1996-2003 @cs.pub.ro)
  - Reteaua de comutatie este de tip Gigaplane XB (102.4GB/s; p2p: 1.6GB/s)
  - Comutare de pachete cu cai separate de adrese si date XB
  - Adresele distribuite printr-un router ce face broadcast
  - Acest fapt permite
    - Partitionarea dinamica a setului de procesoare si crearea unor partitii
    - Aceste partitii sunt complet izolate intre ele si pot fi utilizate ca atare pentru sarcini distincte
  - Suporta paralelism de tip shared-memory in Fortran & C/C++ OpenMP

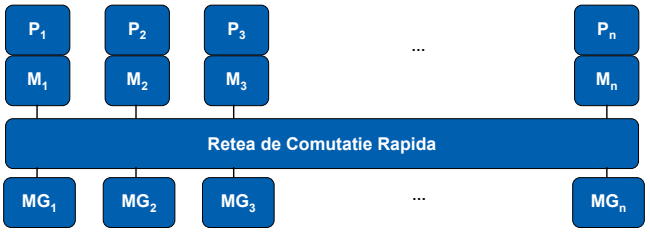




Sisteme Strans / Slab Cuplate

Sisteme Strans / Puternic Cuplate

- Dispun de procesoare P_j cu memorie locala asociata M_i de tip multiport
- O retea de comutatie rapida RC (dar fixa) ce permite accesul uniform la memoria globala partajata MG_i
- Comunicarea si sincronizarea se face prin MG_i cu o rata de transfer similara cu a memoriei locale
- Asigura o scalabilitate redusa datorita faptului ca RC este fixa si de aceea au o structura dedicata
- Performantele sistemului sunt date de modul de acces la memoria partajata

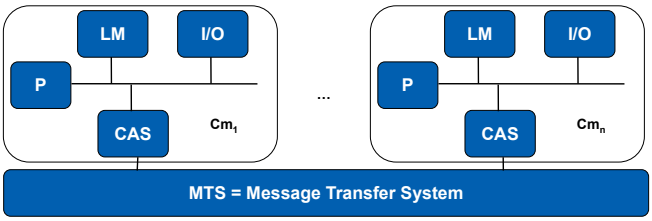


Sisteme Slab Cuplate

- De tip cluster → calculatoarele au resurse locale necesare functionarii independente
- Procesele se executa pe procesoare care dispun de toate resursele pentru implementarea task-ului respectiv
- Comunicarea intre procesoare se face prin
  - Transfer de mesaje
  - Comutare de circuite
- Sistemele slab cuplate (distribuite) sunt eficiente atunci cand interactiunea intre procesoare este minima (granularitatea problemei este mare)
  - Multicalculator centralizate

Sisteme Slab Cuplate

- Resurse locale ale CM_i = Computer Module:
  - LM = local memory (acces din interior/exterior)
  - I/O = magistrala de intrare/iesire (acces din interior/exterior)
  - MTS = Message Transfer System – poate fi implementata cu comutare de circuite sau transfer de mesaje
  - MTS dicteaza performantele prin lungimea mesajelor & viteza de lucru
  - CAS = Collision Avoidance System – legatura si accesul la MTS se fece prin excludere mutuala:
    - Un singur transfer odata daca MTS este magistrala
    - Mai multe transferuri simultane daca MTS este de tip retea



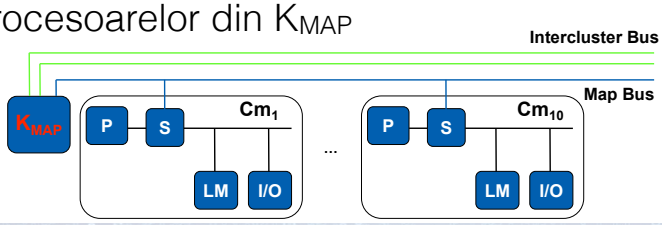
Exemplu – Cm*

- Structura Cm* a fost dezvoltata arhitectural si implementata la Carnegie Mellon in anii ‘70
- Cm* e proiectat cu intentia de a obtine in acelasi timp:
  - Performante bune
  - Fiabilitate sporita
- In 1975 structura e compusa din 10 procesoare
- Ajunge in 1977 la 50 de procesoare
- Cm* beneficiaza de doua sisteme de operare distincte



Arhitectura Cm*

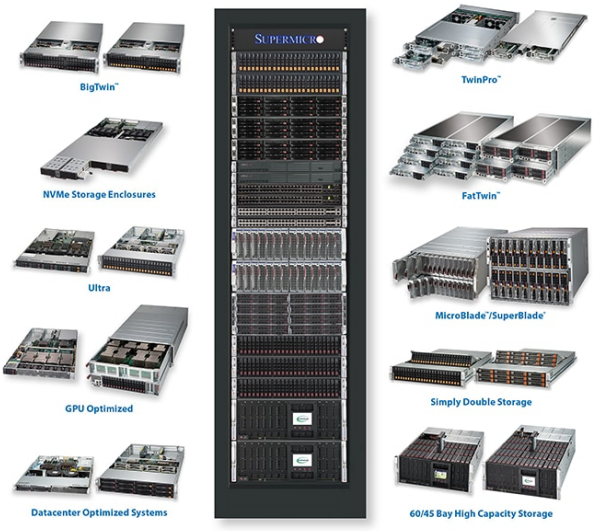
- Structura Cm* conecteaza mai multe module individuale → structura multipla de calculatoare == cluster
- K_{MAP} = gestioneaza magistrala Map Bus si cererile pentru Cm-uri
- Un cluster e format din mai multe Cm-uri, K_{MAP} & Map Bus
- Cm-urile sunt fiabile → asigura semiparalelism
- Performantele sunt determinate de interconectarea clusterelor & viteza de comunicatie a procesoarelor din K_{MAP}



Analogie - Clustere / Servere







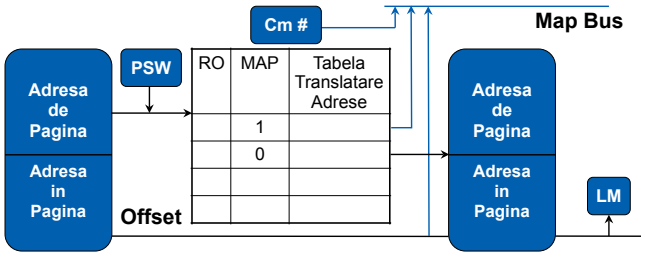


Accesul la Resursele Locale – Switch-ul S

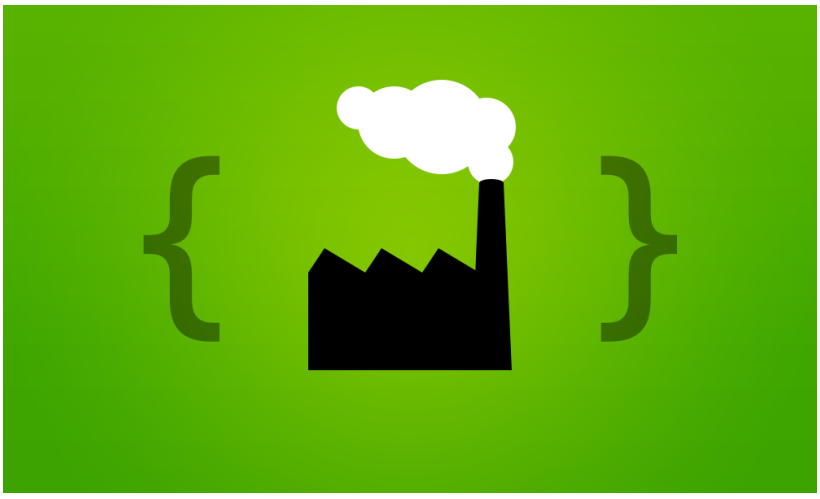
Accesul la Resursele Locale

- Functiile switch-ului S dintr-o structura Cm:
  - Primeste si interpreteaza cererile de acces de la P-uri locale si externe la memoria locala si la sistemul I/O
  - S permite unui P local sa acceseze resurse externe Cm
- Pentru a realiza interpretarea cererilor locale si externe S-ul asigura:
  - O translatie a adreselor la nivel local
  - Formarea adresei externe prin concatenarea adresei in cadrul paginii si numarul de identificare al procesorului
- Memoria este paginata cu dimensiunea paginii de 4K
  - 16 pagini in total → 64K
- Spatiul total de adresare este de 28 de biti - 256M

Mecanismul de Adresare pentru Referinte la Memoria Locala



- PSW = Processor Status Word – ofera adresa in tabela de translatare (adresa fizica/ virtuala)
- RO = Read Only bit (0 = RO; 1 = RW)
- MAP = Maparea adreselor (0 = adresa locala; 1 = adresa globala)
- Cm # = in ce Cm se afla adresa externa; K_{MAP} interpreteaza si decide daca adresa este in cluster-ul curent sau inafara lui



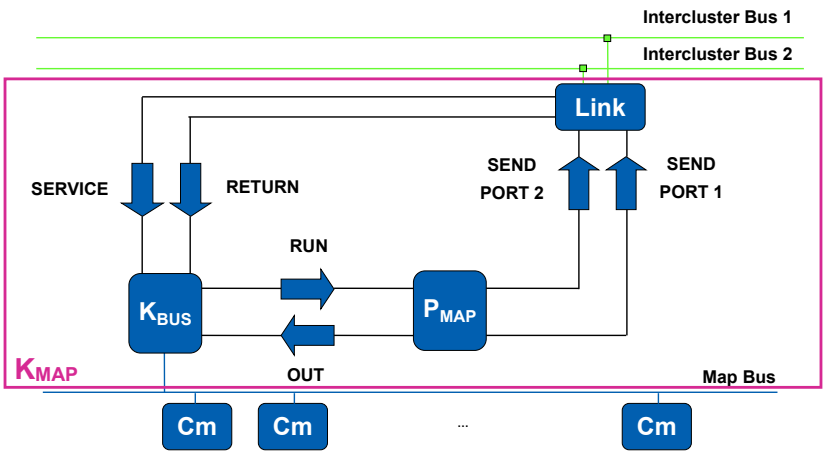
Structura si Functiile K_{MAP}

Functiile K_{MAP}

- K_{MAP} asigura:
  - Controlul in cadrul unui cluster – prin excluderea mutuala a accesului la MapBus si arbitrare intre Cm-urile din acelasi cluster
  - Comunicarea intre clustere prin Intercluster Bus
- Functiile de baza ale K_{MAP}:
  - Mapare de adrese
  - Comunicare
  - Sincronizare
- O serie din primitivele OS sunt implementate in K_{MAP}
- Nucleul OS din Cm se simplifica → transparenta

Structura K_{MAP}

- K_{MAP} este compus din 3 procesoare: P_{MAP}, Link, K_{BUS}:





Elementele K_{MAP}

- $K_{BUS:}$ unitate de comanda a Map Bus si va controla toate tranzactiile de pe aceasta magistrala
- P_{MAP}: procesor de mapare care face toate translatarile de adresa
  - Maxim 8 procese simultan
- Link: gestioneaza si supravegheaza transmiterea mesajelor intercluster Send/Recv
- Exista doua tipuri de comutatie:
  - De circuite – in cadrul cluster-ului
  - De mesaje – intre clustere
- K_{MAP} prelucreaza tranzactii (contexte) concurente
- Fiecare context are asociate registre speciale si registre de legatura la subrutine pt translatarea adreselor



Comunicatia K_{BUS} – P_{MAP}

- K_{BUS} mentine starea a 8 contexte si le aloca cereri de serviciu
  - Fiecare va fi rezident in P_{MAP}
- Numarul contextului si alte informatii sunt plasate in coada RUN
- Din RUN sunt preluate de P_{MAP} care activeaza contextul si eventual translateaza adresa si o plaseaza in OUT
- In acest timp K_{BUS} poate citi alte cereri sau efectua alte functii cerute de P_{MAP}
- In P_{MAP} se executa o schimbare de context prin intermediul celor doua cozi: RUN/OUT



Comunicatia K_{BUS} – P_{MAP}

- K_{BUS} preia acum cererea din OUT si transfera adresa si datele la S-ul local al destinatiei; apoi se ocupa de alte cereri
- Cand accesul la memorie este terminat, K_{BUS} citeste confirmarea de terminare sau citeste datele si plaseaza iara contextul in RUN, reactivandu-l
- K_{BUS} trimite apoi confirmarea la procesul ce a initiat cererea de serviciu
- Contextul poate fi utilizat acum pentru o noua cerere de serviciu



Comunicatia Link – P_{MAP}

- P_{MAP} trimite in SEND o cerere de transmitere mesaj inter-cluster
- Se suspenda contextul initiator al mesajului intercluster & il trece intr-o coada de asteptare
- La primirea raspunsului pentru acest context se pune in RETURN
- Link reactiveaza contextul in RUN
- La primirea raspunsului inter-cluster in Link acesta il plaseaza in SERVICE
- Se emite spre K_{BUS} o cerere de alocare a unui nou context in RUN



Comunicatia K_{BUS} – Map Bus

- K_{BUS} trebuie sa execute o mare varietate de task-uri printre care si tratarea inteligenta a erorilor
- K_{BUS} este un procesor microprogramat cu o memorie de control ROM de 256 de cuvinte a cate 40 de biti fiecare
- Map Bus este o magistrala de comunicatii de 38 de biti dintre care 20 bidirectionali de adrese / date intre K_{BUS} si S-urile locale
- K_{BUS} controleaza toate accesele pe Map Bus precum si cozile si registrele pentru comunicarea cu P_{MAP}
- Maparea Map Bus este sincrona cu K_{BUS}

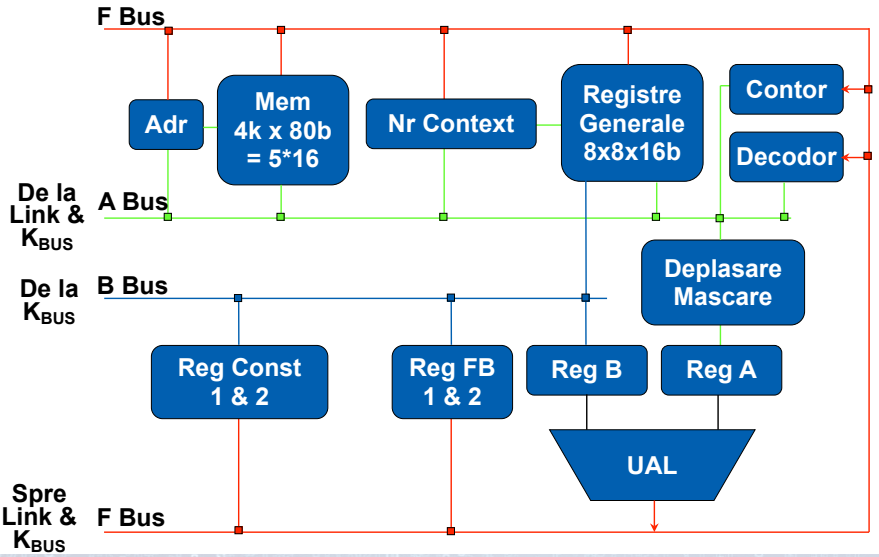


Arhitectura Procesorului de Mapare P_{MAP}

Complexitatea Arhitecturii P_{MAP}

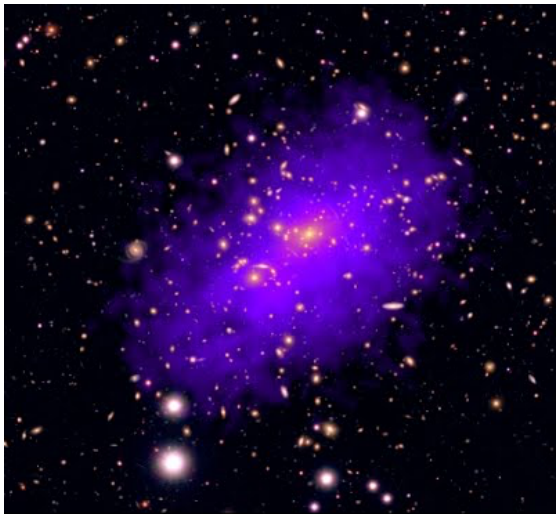
- P_{MAP} este un procesor dedicat pe 16 biti, microprogramat
- Memoria de control este formata din 4k * 80 de biti
- Memoria de control RAM foarte rapida are cuvantul de 80 biti (5*16) – Writable Control Store
- Functiile fundamentale ale procesorului P_{MAP} sunt:
  - Translatarea adreselor nelocale din cluster
  - Implementarea primitivelor OS critice din punctul de vedere al timpului de executie
  - Culegerea informatiilor statistice pentru imbunatatirea solutiilor de proiectare

Arhitectura Procesorului P_{MAP}



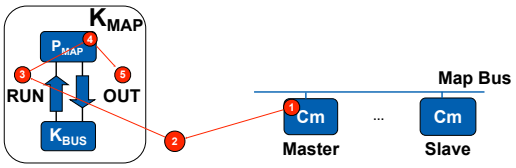
Componentele Procesorului P_{MAP}

- In registrele generale (8x128b sau 8x8x16b) este salvat contextul executiei in curs
- Este organizat in jurul a 3 magistrale:
  - A, B pentru operanzi
  - F pentru rezultate
- Pentru a putea mapa adresele PMAP trebuie sa aibe acces rapid la informatia pt translatarea adreselor virtuale in adrese fizice
- UAL are un operator la nivel de bit pentru Reg A ce asigura
  - Deplasarea
  - Rotirea
  - Mascarea (cu 30 de masti predefinite), permitand preluarea descriptorului de context
- Prelucrarea este in banda de asamblare – operatii UAL si operanzii ce vin simultan, separat, prin registrii tampon
- Accesul la o memorie in acelasi cluster este mult mai rapid decat in alt cluster



Comunicarea in Cadrul unui Cluster

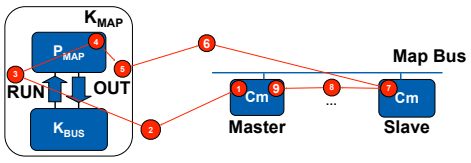
Etapele de Acces
in Cadrul unui Cluster



1. In Cm Master se initiaza acces la o memorie nelocala
2. Cm Master emite adresa virtuala catre K_{BUS}
3. K_{BUS} activeaza un context (crearea structurii de date specifica tranzitiei) ce se pune in coada RUN catre P_{MAP}
4. P_{MAP} trateaza contextul si face translatarea adresei
5. P_{MAP} pune in coada OUT o cerere pentru ciclu de memorie pentru Cm Slave din cluster-ul curent



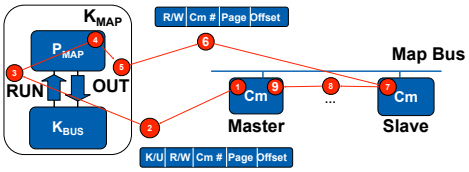
Etapele de Acces
in Cadrul unui Cluster



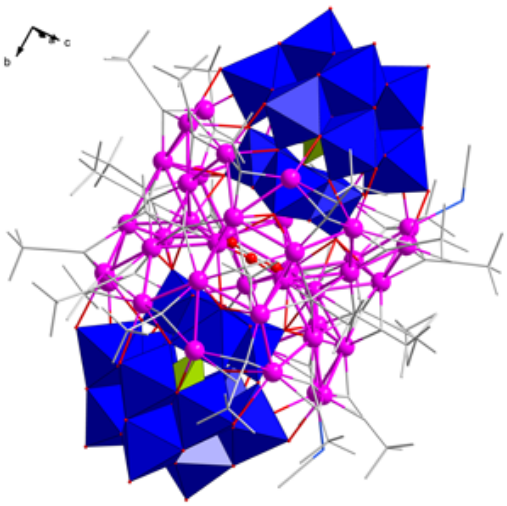
6. K_{BUS} trimite adresa fizica catre Cm Slave prin Map Bus
7. S-ul local din Cm Slave face acces la memoria locala prin furt de ciclu
8. K_{BUS} “permite” ca rezultatul operatiei de acces la memorie sa fie furnizat catre Cm Master
9. Cm Master preia datele, termina operatia in curs si continua executia



Formatul Pachetelor Transmise

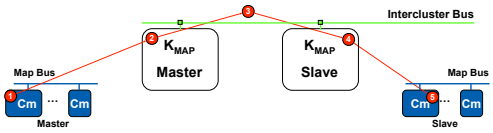


- Pachetele trimise au urmatorul format:
  - U/K – User/Kernel
  - R/W – Read/Write
  - Cm # – numarul Cm-ului ce efectueaza transferul
  - Page – utilizata pentru translatarea locala
  - Offset – In cadrul paginii, adresa relativa
- Memoriile Cm-urilor trebuie sa aiba posibilitatea de dublu acces



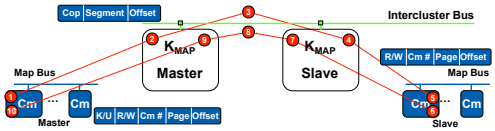
Comunicarea Intercluster

Comunicarea Intercluster



1.Cm Master trimite o cerere de transfer la K_{MAP} Master
2.K_{MAP} Master pregateste mesaj/pachet intercluster si codifica cererea
3.Mesajul intercluster e transmis pe Intercluster Bus pe baza algoritmilor de rutare
4.K_{MAP} Slave decodifica cererea primita si o trimite catre Clusterul sau local
5.Cererea de ciclu de memorie este trimisa la Cm Slave

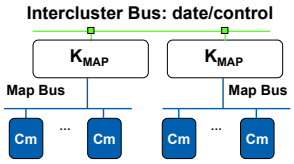
Comunicarea Intercluster



6. Cm Slave transmite rezultatul catre K_{MAP} Slave
7. K_{MAP} Slave pregateste mesajul intercluster - reactivarea contextului respectiv
8. K_{MAP} Slave transmite rezultatul catre K_{MAP} Master
9. K_{MAP} Master receptioneaza si interpreteaza mesajul primit
10.Rezultatul este trimis catre Cm Master

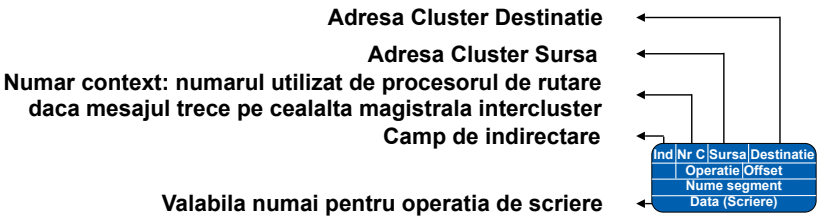
Protocolul Magistralei Intercluster

- Magistrala intercluster
  - Contine linii de date si control
  - Nu contine linii de adresa pt ca se realizeaza comutare de pachete si nu de circuite
- Comunicarea este:
  - Asicrona
  - De tip intrebare-raspuns (Q & A)
  - Cu interblocare completa – pentru fiecare actiune se primeste raspuns si abia apoi se anuleaza comanda
- Mesajele intercluster sunt formate din 1-8 cuvinte
- Cele mai utilizate: mesajele directe si de raspuns

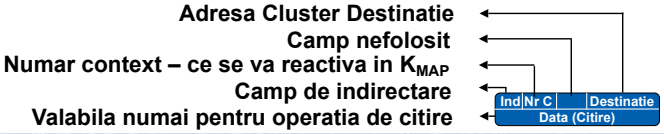


Formatul Mesajelor Intercluster

- Mesajul Direct – format din 4 cuvinte



- Mesajul de Raspuns – format din 2 cuvinte



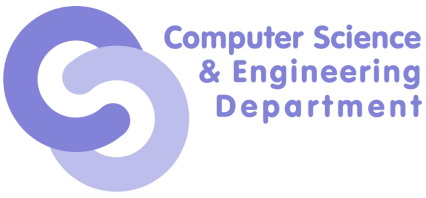
Ce am invatat in aces curs?

- Caracteristicile arhitecturilor MIMD
- Sisteme Multiprocesor / Multicalculator
- Sisteme cu Memorie Partajata
- Sisteme cu Memorie Distribuita
- Sisteme Strans / Slab Cuplate
- Accesul la Resursele Locale – Switch-ul S
- Structura si Functiile K_{MAP}
- Arhitectura Procesorului de Mapare P_{MAP}
- Comunicarea in cadrul unui Cluster
- Comunicarea Intercluster



Thank you for your attention.





emil.slusanschi@cs.pub.ro
cs.pub.ro